

Session 4: Random spawning & Input handling

#1 Random Piece spawning

```
int nextPieces[5] = {-1};
```

다음 블록 리스트 global variable 로 정의

```
void SpawnNextPiece() {
    static int bag[7] = {0, 1, 2, 3, 4, 5, 6};
    static int bag_index = 7;

    if (nextPieces[0] == -1) bag_index = 7; // Force shuffle on first call

    if (bag_index >= 7) {
        bag 섞기
    }

    if (nextPieces[0] == -1) {
        랜덤으로 초기값 설정
    } else {
        하나씩 앞으로 당기고 마지막은 bag 에서 가져오기
    }

    currentPieceX = 3;
    currentPieceY = 0;
    currentPieceRotation = 0;
}
```

테트리스 게임에서는 블록들이 완전히 랜덤으로 스폰되는 것이 아니라 7 bag system 을 통해서 스폰됨. 이렇게 하면 특정 블록이 집중적으로 나오거나 같은 블록이 3번 이상 연속으로 나오는 경우가 제외됨.

InitializeGame 함수와 메인 루프에서 블록 스폰시키는 부분을 해당 함수로 대체함. 메인 루프에서는 스폰 전에 충돌 확인하고 충돌하면 재시작.

#Optional block holding & drawing

```
int holdPiece = -1;
int holdUsed = 0; // can swap once
```

홀드 시스템을 위한 global variable

```
void SwapHoldPiece() {
    if (holdPiece == -1) {
```

```

        holdPiece = currentPieceType;
        SpawnNextPiece();
    } else {
        int temp = holdPiece;
        holdPiece = currentPieceType;
        currentPieceType = temp;
        currentPieceX = 3;
        currentPieceY = 0;
        currentPieceRotation = 0;
    }
}

```

홀드 동작 함수 정의

InitializeGame 함수에 초기화하는 코드 추가함

```

void DrawNextPiecesAndHoldPiece() {
    for (int i = 0; i < 5; i++) {
        nextPiece array 에 있는 블록들 오른쪽에 그리기
    }
    if (holdPiece != -1) {
        홀드하고 있는 블록 왼쪽에 그리기
    }
}

```

화면에 그리는 함수 정의, 메인 루프에 추가

#2 Input handling

```
int last_fall_time;
```

global variable로 바꿈, InitializeGame 함수에 초기화하는 코드 추가함

```

switch (e.key.keysym.sym) {
    case SDLK_ESCAPE: // Quit on Escape key
        running = false;
        break;
    case SDLK_LEFT:
        충돌 확인하고 왼쪽으로 이동
    case SDLK_RIGHT:
        충돌 확인하고 오른쪽으로 이동
    case SDLK_DOWN:
        충돌 확인하고 아래쪽으로 이동
    case SDLK_UP:
    case SDLK_x:

```

```

        충돌 확인하고 시계방향으로 회전
case SDLK_z:
    충돌 확인하고 반시계방향으로 회전
case SDLK_c:
case SDLK_LSHIFT:
    if (!holdUsed) {
        if (holdPiece == -1) {
            SwapHoldPiece();
            holdUsed = 1; // can't use again
            last_fall_time = SDL_GetTicks();
        } else {
            충돌 확인하고 Swap
        }
        break;
case SDLK_r:
    InitializeGame(); // Restart the game
    break;
case SDLK_SPACE:
    하드 드롭 (바로 바닥으로 떨어지게, 충돌하기 전까지 y++)
    last_fall_time = -BLOCK_FALL_INTERVAL; // force lockPiece
    break;
}

```

입력된 키에 따라 동작하게 만들.